CS-GY 6613
Assignment 2
2026-02-22
Chan Young Kang

Image-to-Video Semantic Retrieval via Object Detection

We implemented a detection-driven visual retrieval system that identifies car exterior components in a video and retrieves temporally coherent video segments given a query image. The system is built around a YOLOv8 segmentation model and converts raw video into structured semantic segments through detection, indexing, and rule-based matching.

We selected YOLOv8 segmentation (Ultralytics) as the object detection framework because it supports part-level instance segmentation, outputs bounding boxes, class labels, and confidence scores directly, and integrates cleanly into Python pipelines. We fine-tuned the pretrained yolov8n-seg.pt model on the Ultralytics Car Parts Segmentation dataset. This dataset contains labeled masks for car exterior components and is formatted in YOLO segmentation format with a dataset YAML configuration file.

To better understand the effect of continued training, we compared performance at 30 epochs and after extending training to 50 epochs. Figures 1 and 2 show the full training curves for both configurations. From 30 to 50 epochs, training and validation losses continued to decrease slightly, while mAP@50 and mAP@50–95 improved marginally before stabilizing. The precision and recall curves show reduced volatility in later epochs, indicating improved model stability. Importantly, validation loss tracked training loss closely in both configurations. No divergence between training and validation curves was observed, suggesting that overfitting did not occur. However, the improvement from 30 to 50 epochs was incremental rather than dramatic, indicating diminishing returns beyond 30 epochs. For this reason, we selected the 50-epoch checkpoint (best_50ep.pt) as the final model, as it provided the strongest overall metrics without unnecessary additional training. Because CPU training was too slow, training was performed on Google Colab using a T4 GPU.

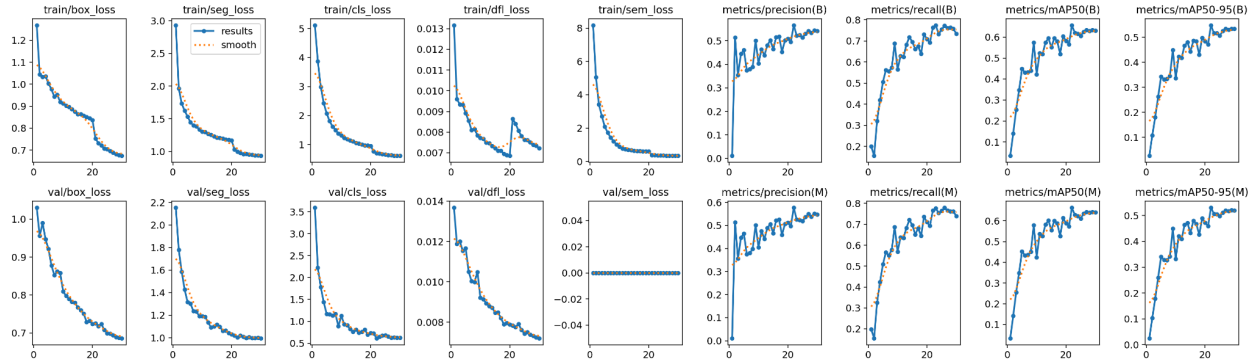

Figure 1: Training metrics at 30 epochs

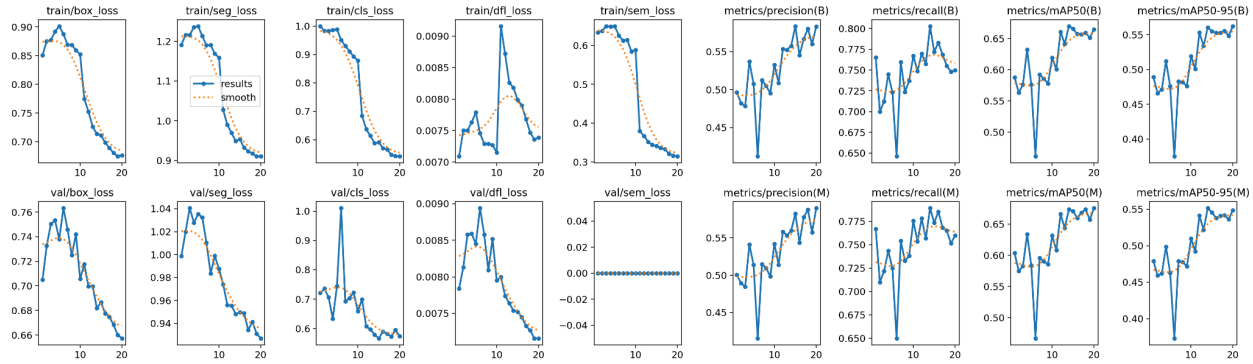

Figure 2: Training curves after extending training to 50 epochs

For video processing, we used a Toyota RAV4 review video (YouTube ID: YcvECxtXoxQ). The video was sampled at one frame per second (1 FPS). This sampling rate reduced computational cost while preserving sufficient temporal resolution for object presence detection. It also prevented overly fragmented retrieval clips that would occur with higher sampling rates. Frames were extracted using ffmpeg and stored sequentially.

We ran YOLOv8 segmentation inference on each sampled frame using the trained 50-epoch model. For each frame, we extracted the detected component class labels, confidence scores, and bounding boxes. A confidence threshold of 0.30 was applied to reduce noisy detections and improve retrieval precision.

Additional filtering was introduced to suppress false detections. During qualitative inspection, we observed that a presenter's legs were occasionally misclassified as wheels due to similar circular and high-contrast visual patterns. To address this, we used a pretrained COCO detector to identify persons in each frame. If a detected wheel significantly overlapped with a person bounding box, the wheel detection was removed. Figures 3 and 4 illustrate this improvement: before filtering, a wheel is incorrectly detected on the presenter's legs; after filtering, the false detection is removed while valid vehicle components remain intact. This post-processing step improved precision and overall retrieval correctness without harming detection of legitimate car parts.

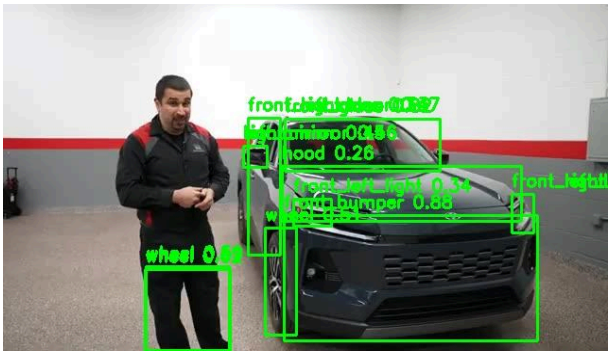

Figure 3: Raw Detection Without Filtering

Figure 4: Detection After Car–Person Filtering

The detection outputs were stored in a frame-level Parquet file named detections_50ep_framelevel.parquet. Following professor guidance, each row corresponds to a single frame and contains a nested list of detection dictionaries. Each detection includes class_label, confidence_score, and bounding_box. This file serves as the sole interface between the detection and retrieval stages of the pipeline.

Query images were obtained from the Hugging Face dataset aegean-ai/rav4-exterior-images, which contains 65 RAV4 exterior images. For each query image, we ran the same YOLOv8 segmentation model and extracted detected component labels. We verified that 62 out of 65 query images produced at least one detection. Label normalization was applied consistently across query and video detections to prevent mismatch errors. Query detections were optionally filtered using a top-k strategy to retain only the highest-confidence component labels.

The retrieval system operates by matching query labels to frame-level detections. For each video frame, we compute the overlap between the set of query labels and the set of labels detected in that frame. A frame is considered a match if the number of overlapping labels meets a required threshold defined as:

need = max(min_need, ceil(min_fraction × number_of_query_labels)).

After extensive experimentation, the final retrieval configuration used was:

--topk 3
--min-need 2
--min-fraction 0.6
--min-segment-sec 3

Using --topk 3, we retained only the three highest-confidence labels from each query image. This reduced noisy matches and prevented trivial components (such as wheels, which appear in many frames) from dominating retrieval results. The parameter --min-need 2 required that at least two query components appear in a frame for it to qualify as a match. The parameter --min-fraction 0.6 enforced that at least 60% of the query labels must match, making the retrieval rule adaptive to query complexity. The parameter --min-segment-sec 3 required that matching frames persist for at least three consecutive seconds before being returned as a segment, ensuring temporal coherence and preventing short, fragmented clips.

Instead of returning individual matching frames, we grouped consecutive matching frames into contiguous temporal segments. Matching frames were sorted by timestamp and aggregated into segments whenever adjacent timestamps were within the allowed gap threshold. Each segment was then converted into a start and end timestamp corresponding to a YouTube clip (e.g., ?start=120&end=165). This smoothing step reduces frame-to-frame noise and ensures that retrieved results correspond to coherent video portions rather than isolated detections.

The final retrieval output was stored in retrieval_segments.parquet. Each row represents a single query–segment pair and includes video_id, query_index, query_labels, start_timestamp, end_timestamp, class_label, number_of_supporting_detections, and youtube_url. The class_label indicates which matched component the segment is attributed to, and number_of_supporting_detections represents the total number of detections of that component across all frames within the segment. Because multiple detections can occur per frame, this count may exceed the number of frames in the segment.

During development, we encountered and resolved several important issues. Initially, requiring all query labels to match resulted in zero segments. This was corrected by introducing min_need, min_fraction, and topk to allow partial but meaningful overlap. We also observed over-concentration in simple queries containing common components such as front glass and wheels. To diagnose this, we implemented a segment concentration analysis tool that computed segment counts per query and a Gini-like concentration metric. This allowed us to tune parameters to reduce trivial matches and improve semantic relevance.

Throughout optimization, we prioritized precision over recall. Because grading emphasizes correctness, specifically whether retrieved clips truly contain the queried component, we accepted missing some frames in exchange for cleaner, more reliable segments. Requiring multiple overlapping labels and enforcing minimum segment duration significantly improved retrieval quality and temporal coherence.

The final system is a fully reproducible detection-driven retrieval pipeline. It fine-tunes a segmentation model, performs scalable video inference, stores structured detection outputs, matches query labels to video frames using explicit overlap rules, aggregates frames into temporally coherent segments, and outputs clean retrieval results in Parquet format. The system is deterministic, tunable, structurally clean, and directly aligned with the assignment's grading criteria for correctness, temporal coherence, detection quality, and data engineering quality.